CODING-FIRST · SENIOR SWE L3 · CONFLUENT KORA — MARKETPLACE & HUB

Confluent Prep — Problems, Poore Solutions, Har Sawal Ka Jawab

Har problem: pehle normal solution (comments ke saath), fir complexity, fir multithreading. Jahan file-system / threading / exception ka concept aayega, wahin samjha diya jaayega.

Padhne ka tarika: Har problem 4 steps me hai — **STEP 1 SAMJHO** **STEP 2 SIMPLE SOLUTION** **STEP 3 COMPLEXITY/THREADING** **STEP 4 TRAPS + Q&A**

Narangi box = CONCEPT BREAK — jahan koi bhaari cheez (thread pool, RandomAccessFile, exception handling) pehli baar aayegi, wahin uska concept khol ke samjhaya gaya hai. Har question ka **poora answer** diya gaya hai, sirf hint nahi.

ROLE REMINDER — SCOPE CLEAR RAKHO

Senior SWE (L3), **KORA ke andar Marketplace & Hub domain**. Kaam: Kafka Connect connectors aur Flink artifacts ka **catalog, publishing, distribution, lifecycle management** — matlab connector "App Store" ko power karne wali services. Poore KORA broker-internals pe nahi. Bangalore, remote, Java/Scala/Go — tu Java me rahega.

ABHI KA STAGE

HackerRank Coding Assessment (Round 0). Ye clear hone ke baad hi 5 rounds shuru honge — isliye Chapter 01-03 (coding) abhi sabse important hai, Chapter 04 (HLD/ Domain/Behavioral) thoda baad me bhi chalega.

Chapter	Kya milega
01 Concurrency Problems	Sudoku Validator, Job Scheduler, Ring Buffer + threading concepts inline
02 LRU	Classic → TTL → Heaps+Maps → Thread-safe version
03 DSA + LLD	

	Session Manager, Rate Counter, Number of Atoms, tail -f, Large File, Map+Tree
04 HLD + Domain + Behavioral	Requirement framework, KV Store, Kafka internals, KORA, Marketplace, STAR

CHAPTER 01

Concurrency Problems

Har problem pehle single-thread me solve hoti hai, fir threading add hoti hai — aur threading ka har naya tool wahin samjhaya jaata hai.

1 Sudoku Validator — Multithreaded

STEP 1 — PROBLEM SAMJHO

9x9 board valid hai ya nahi. Rules: har row me 1-9 unique, har column me 1-9 unique, har 3x3 box me 1-9 unique. Kul **27 checks** *(9 rows + 9 cols + 9 boxes). Sabse badi baat: ye 27 checks ek doosre pe depend nahi karte — koi bhi check kisi doosre ka result nahi maangta.*

STEP 2 — NORMAL (SINGLE-THREAD) SOLUTION

PEHLE YAHI LIKHO, YE HAMESHA KAAM KAREGA

```
class SudokuValidator {
    private final int[][] board;
    // 'final' = reference ek baar set hone ke baad badla nahi ja sakta.
    // Aage jab threads aayenge, ye immutability hi safety degi.

    public SudokuValidator(int[][] board) {
        this.board = board;
    }

    public boolean isValidSequential() {
        for (int i = 0; i < 9; i++) {
            if (!checkRow(i))  return false;   // pehla fail milte hi ruk
jao
            if (!checkCol(i))  return false;
            if (!checkBox(i))  return false;
        }
        return true;
    }

    private boolean checkRow(int r) {
        boolean[] seen = new boolean[10];   // index 1..9 use karenge, 0
waste
        for (int c = 0; c < 9; c++) {
            int val = board[r][c];
            if (val == 0) continue;         // 0 = khaali cell, skip
            if (seen[val]) return false;    // ye number pehle aa chuka =
```

```
duplicate
            seen[val] = true;
        }
        return true;
    }

    private boolean checkCol(int c) {
        boolean[] seen = new boolean[10];
        for (int r = 0; r < 9; r++) {
            int val = board[r][c];
            if (val == 0) continue;
            if (seen[val]) return false;
            seen[val] = true;
        }
        return true;
    }

    private boolean checkBox(int b) {
        boolean[] seen = new boolean[10];
        // Box 0..8 hote hain, left-to-right phir top-to-bottom.
        // Box b ka top-left cell nikalne ka formula:
        int startRow = (b / 3) * 3;   // b=4 -> 4/3=1 -> row 3
        int startCol = (b % 3) * 3;   // b=4 -> 4%3=1 -> col 3
        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {
                int val = board[r][c];
                if (val == 0) continue;
                if (seen[val]) return false;
                seen[val] = true;
            }
        }
        return true;
    }
}
```

Complexity: har cell 3 baar dekha jaata hai (row check, col check, box check) → O(81) = O(1) for fixed 9x9. Space O(1).

STEP 3 — AB MULTITHREADING ADD KARO

KYUN ADD KAREIN — AUR KYUN YE SAFE HAI

27 tasks hain jo ek doosre se independent hain. Aur sabse important: **koi bhi task board ko modify nahi karta** — sab sirf **read** kar rahe hain. Har task apna alag `seen[]` array banata hai (wo local variable hai, thread ka apna stack pe rehta hai).

Rule yaad rakho: read-only / immutable shared data **apne aap thread-safe** hota hai. Isliye yahan koi lock, koi synchronized nahi chahiye.

CONCEPT BREAK — EXECUTORSERVICE, CALLABLE, FUTURE

```
// 1. Thread pool banao. Har task ke liye naya Thread banana mehnga hai;
//    pool me kuch fixed threads hote hain jo tasks ko reuse karke chalate
hain.
ExecutorService pool = Executors.newFixedThreadPool(9);

// 2. Runnable vs Callable:
//    Runnable  -> run(),  kuch return nahi karta, checked exception throw
nahi kar sakta
//    Callable  -> call(), value return karta hai AUR exception throw kar
sakta hai
//    Humein har check ka true/false chahiye, isliye Callable<Boolean>.

// 3. Future<T> = "abhi result nahi hai, baad me milega" ka placeholder.
//    future.get() BLOCKING hai - result aane tak current thread wahin ruk
jaata hai.

// 4. invokeAll(tasks) = saare tasks submit karo aur SAB complete hone tak
wait karo.
//    Ye khud blocking hai, isliye iske baad results ready hote hain.

// 5. pool.shutdown() zaroori hai. Warna pool ke threads non-daemon hote
hain
//    aur JVM exit hi nahi karega. Isliye finally block me daalte hain.
```

MULTITHREADED VERSION

```
public boolean isValid() throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(9);
    List<Callable<Boolean>> tasks = new ArrayList<>();

    for (int i = 0; i < 9; i++) {
        final int idx = i;
        // 'final' (ya effectively-final) hona zaroori hai:
        // lambda apne bahar ke local variable ko tabhi capture kar sakta
hai
        // jab wo badalta na ho. Loop variable 'i' badalta hai, isliye copy
banayi.
        tasks.add(() -> checkRow(idx));   // ye Callable<Boolean> ban gaya
        tasks.add(() -> checkCol(idx));
        tasks.add(() -> checkBox(idx));
    }

    try {
        List<Future<Boolean>> results = pool.invokeAll(tasks);
        for (Future<Boolean> f : results) {
            if (!f.get()) return false;   // .get() = us task ka actual
result
```

```
        }
        return true;
    } finally {
        pool.shutdown();   // exception aaye ya na aaye, pool band hona hi
chahiye
    }
}
```

FOLLOW-UP JO PAKKA AAYEGA — "EARLY EXIT KAISE KAROGE?"

Upar wala `invokeAll` saare 27 tasks poore hone ka wait karta hai, chahe pehla hi invalid nikal jaaye. Fix: ek shared `AtomicBoolean` flag.

```
// AtomicBoolean = ek boolean jise multiple threads bina lock ke safely
// padh/likh sakte hain (andar CAS - compare-and-swap - use hota hai).
// Plain 'boolean' yahan galat hota: ek thread ki likhi value doosre thread
ko
// turant dikhegi iski koi guarantee nahi hoti (CPU cache ki wajah se).
AtomicBoolean invalid = new AtomicBoolean(false);

Callable<Boolean> task = () -> {
    if (invalid.get()) return false;   // koi aur already fail kar chuka,
mat chalo
    boolean ok = checkRow(idx);
    if (!ok) invalid.set(true);        // sabko bata do ki board invalid hai
    return ok;
};
```

Dusra tarika: `CompletionService` — ye tasks ke results **jaise-jaise aate hain** waise deta hai, submission order me nahi. Pehla `false` milte hi tum ruk sakte ho.

STEP 4 — TRAPS AUR POORE ANSWERS

Interview me ye bolna: "27 checks independent hain aur board read-only hai, isliye koi synchronization nahi chahiye. Sirf early-exit flag ke liye AtomicBoolean use karunga."

REALITY CHECK JO BOLNE SE POINTS MILTE HAIN

9x9 board ke liye multithreading actually **slower** ho sakti hai — thread pool banane ka overhead hi kaam se zyada hai. Ye khud bol dena: "9x9 pe main sequential hi rakhunga; multithreaded design tab meaningful hai jab board bada ho (16x16, 25x25) ya hazaaron boards batch me validate karne hon." Interviewer ko lagega tu sirf pattern nahi likh raha, soch raha hai.

Q1: Is problem me koi lock kyun nahi chahiye?

A: Kyunki koi bhi thread board ko modify nahi kar raha — sab sirf read kar rahe hain. Race condition tabhi hoti hai jab shared data **mutable** ho. Yahan board read-only hai aur har task ka `seen[]` array uska apna local variable hai (thread ke apne stack pe), shared nahi. Isliye synchronization ki zaroorat hi nahi.

Q2: count++ jaisi cheez atomic kyun nahi hoti? (ye follow-up aksar yahin se nikalta hai)

A: `count++` dikhne me ek line hai, but CPU ke liye teen operations hain — **read → add → write**. Do threads beech me ghus sakte hain: dono purani value 5 padh lete hain, dono 6 banake likh dete hain, aur ek update kho jaata hai. Isliye ise "non-atomic operation" kehte hain. Fix: `AtomicInteger.incrementAndGet()` ya `synchronized`.

Q3: volatile aur AtomicBoolean me kya farak hai?

A: `volatile` sirf **visibility** deta hai — ek thread ne value badli to doosre ko turant dikhegi, CPU cache se purani value nahi milegi. Lekin ye atomicity nahi deta. `AtomicBoolean`/`AtomicInteger` visibility **aur** atomicity dono dete hain (andar CAS use karke). Isliye simple flag ke liye volatile chalega, but read-modify-write (jaise increment ya compareAndSet) ke liye Atomic chahiye.

Q4: Runnable aur Callable me kya farak hai?

A: `Runnable.run()` kuch return nahi karta aur checked exception throw nahi kar sakta. `Callable.call()` value return karta hai (`Future` ke through milti hai) aur checked exception throw kar sakta hai. Jab task ka result chahiye ya exception propagate karni ho, Callable use karo.

2 Job Scheduler with Concurrency

STEP 1 — PROBLEM SAMJHO

Ek scheduler banao jahan log tasks submit kar sakein — kuch turant chalein, kuch 5 minute baad, kuch har ghante repeat hon. Multiple threads ek saath submit kar rahe hain, aur multiple workers execute kar rahe hain.

DESIGN KE 4 CORE COMPONENTS (YE STRUCTURE BOLKE SHURU KARO)

1. **Job** — task ka data (id, runnable, scheduled time, priority, recurring interval)
2. **Priority Queue** — jobs time ke order me, jo pehle chalna hai wo top pe
3. **Worker pool** — jobs ko actually execute karne wale threads
4. **Dispatcher thread** — queue se due jobs uthake pool ko deta hai

STEP 2 — NORMAL (SINGLE-THREAD) SOLUTION

PEHLE SABSE SIMPLE VERSION

```
class Job implements Comparable<Job> {
    String jobId;
    Runnable task;          // actual kaam
    long scheduledTime;     // epoch millis - kab chalna hai
    long intervalMillis;    // 0 = one-time, >0 = har itne ms baad repeat
    int priority;

    // Comparable implement karne se PriorityQueue ko pata chalta hai
    // ki "chhota" kaun hai - aur wahi top pe rahega.
    public int compareTo(Job other) {
        int c = Long.compare(this.scheduledTime, other.scheduledTime);
        if (c != 0) return c;                              // pehle time se
        return Integer.compare(other.priority, this.priority);
        // ulta compare = zyada priority pehle (descending)
    }
}

// Sabse simple scheduler: ek hi thread sab kuch karta hai
class SimpleScheduler {
    private final PriorityQueue<Job> queue = new PriorityQueue<>();

    public void submit(Job job) { queue.offer(job); }

    public void runLoop() throws InterruptedException {
        while (true) {
            Job job = queue.peek();
            if (job == null) { Thread.sleep(50); continue; }
            long delay = job.scheduledTime - System.currentTimeMillis();
            if (delay > 0) { Thread.sleep(delay); continue; }
            queue.poll().task.run();   // yahin, isi thread pe chalao
        }
    }
}
```

Ye kyun kaafi nahi hai: (1) job chalte waqt loop block ho jaata hai — agli job late chalegi, (2) `PriorityQueue` thread-safe nahi, do submitters aaye to corrupt ho sakti hai, (3) ek job exception phenke to poora loop mar jaayega.

STEP 3 — MULTITHREADING ADD KARO

CONCEPT BREAK — BLOCKINGQUEUE AUR VOLATILE

```
// PriorityBlockingQueue = PriorityQueue + thread-safety + blocking take()
//
//   queue.offer(job)  -> daalo (ye kabhi block nahi karta, unbounded hai)
//   queue.take()      -> nikaalo; agar queue KHAALI ho to ye thread wahin
//                        RUK jaata hai jab tak koi element na aaye.
```

```
//                        Isse "busy loop" (baar-baar check karo) se bach
jaate hain.
//
// volatile boolean running:
//   Dispatcher thread is flag ko loop me padhta hai, shutdown() doosre
thread
//   se ise false karta hai. Bina 'volatile' ke dispatcher thread purani
value
//   apne CPU cache me pakde rehta aur kabhi rukta hi nahi.
//   volatile = "ye variable hamesha main memory se padho, cache se nahi".
```

MULTITHREADED SCHEDULER — POORA CODE

```
class JobScheduler {
    private final PriorityBlockingQueue<Job> queue
        = new PriorityBlockingQueue<>();          // thread-safe priority
queue
    private final ExecutorService workers;         // jobs yahan chalengi
    private final Thread dispatcher;               // ye sirf jobs baantega
    private volatile boolean running = true;       // shutdown flag

    public JobScheduler(int workerCount) {
        this.workers = Executors.newFixedThreadPool(workerCount);
        this.dispatcher = new Thread(this::dispatchLoop);
        this.dispatcher.start();
    }

    public void submit(Job job) {
        queue.offer(job);
        // koi lock nahi lagaya - PriorityBlockingQueue khud thread-safe hai
    }

    private void dispatchLoop() {
        while (running) {
            try {
                Job job = queue.take();   // khaali ho to yahin block ho jao
                long delay = job.scheduledTime - System.currentTimeMillis();
                if (delay > 0) {
                    queue.offer(job);     // abhi time nahi hua, wapas daal
do
                    Thread.sleep(Math.min(delay, 100));
                    continue;
                }
                workers.submit(() -> runJob(job));
                // dispatcher khud job nahi chalata - worker ko de deta hai,
                // taaki wo turant agli job dekhne ja sake
            } catch (InterruptedException e) {
                // Koi is thread ko interrupt() kar raha hai (shutdown se).
                // Best practice: flag wapas set karo aur loop se nikal jao.
                Thread.currentThread().interrupt();
```

```
                break;
            }
        }
    }

    private void runJob(Job job) {
        try {
            job.task.run();
        } catch (Exception e) {
            // Ye catch CRITICAL hai. Ek job ki exception se
            // poora scheduler nahi marna chahiye. Log karo, aage badho.
        } finally {
            // finally = chahe job pass ho ya fail, reschedule to karna hi
hai
            if (job.intervalMillis > 0 && running) {
                job.scheduledTime = System.currentTimeMillis()
                                    + job.intervalMillis;
                queue.offer(job);
            }
        }
    }

    public void shutdown() {
        running = false;
        dispatcher.interrupt();   // agar take() pe block hai to use jagao
        workers.shutdown();
    }
}
```

STEP 4 — TRAPS AUR POORE ANSWERS

SABSE BADA TRAP — BUSY-WAIT

Upar wale loop me job ka time nahi hua to hum wapas queue me daal ke 100ms sleep karte hain — matlab thoda-thoda polling ho rahi hai. **Behtar answer:** `DelayQueue` use karo — element tabhi milta hai jab wo "due" ho jaaye, warna `take()` khud block rehta hai. Ya seedha `ScheduledThreadPoolExecutor` (`scheduleAtFixedRate` / `scheduleWithFixedDelay`) jo ye sab built-in deta hai.

Q1: PriorityBlockingQueue kyun, simple PriorityQueue kyun nahi?

A: Do wajah. Pehla — `PriorityQueue` thread-safe nahi hai; multiple submitters aur dispatcher ek saath access karenge to internal heap corrupt ho sakta hai. Doosra — `PriorityBlockingQueue.take()` queue khaali hone pe **block** karta hai, isliye dispatcher ko busy-loop me CPU jalane ki zaroorat nahi padti.

Q2: Ek job crash ho jaaye to?

A: Har job ko `try-catch` me wrap karo (upar `runJob` me kiya hai). Exception ko log karo, job ko ek retry/dead-letter queue me daalo, aur scheduler chalta rahe. Agar tum `pool.submit()` ko bina catch ke chhod do to exception `Future` ke andar chup-chaap dab jaati hai aur kisi ko pata bhi nahi chalta — ye khud se bolna interviewer ko impress karta hai.

Q3: Same job do baar na chale, ye guarantee kaise doge?

A: Ek `ConcurrentHashMap<String,Boolean> runningJobs` rakho. Job chalane se pehle `runningJobs.putIfAbsent(jobId, true)` karo — agar ye `null` return kare tabhi tumne "claim" kiya, chalao; warna koi aur already chala raha hai, skip karo. Job khatam hone pe `finally` me `remove(jobId)`. `putIfAbsent` isliye, kyunki `if (!map.containsKey(k)) map.put(k,v)` do alag steps hain aur beech me doosra thread ghus sakta hai — `putIfAbsent` atomic hai.

Q4: Multi-machine pe kaise scale karoge?

A: In-memory queue ek machine tak seemit hai. Do machines same job utha lengi. Solution: jobs ko shared store me rakho (DB ya Redis) aur job "claim" karne ke liye distributed lock lo — Redis `SET key value NX PX ttl`, ya Zookeeper ephemeral node, ya DB me `SELECT ... FOR UPDATE SKIP LOCKED` se row-level claim. TTL zaroori hai: agar claim karne wali machine mar jaaye to lock apne aap chhoot jaana chahiye, warna wo job hamesha ke liye atak jaayegi.

Q5: Worker pool ka size kaise decide karoge?

A: Job ki nature pe depend karta hai. CPU-bound jobs (calculation) ke liye pool size ≈ core count — usse zyada threads sirf context-switching ka overhead badhaayenge. IO-bound jobs (network/DB calls) ke liye zyada threads rakh sakte ho kyunki threads zyadatar wait me rehte hain. Aur pool bounded rakhna — unbounded `newCachedThreadPool()` load spike me hazaaron threads bana ke system gira sakta hai.

3 Thread-Safe Ring Buffer (O(1) queue + random)

STEP 1 — PROBLEM SAMJHO

Ek queue chahiye jisme **enqueue O(1)**, **dequeue O(1)**, *aur* **random() O(1)** *— teeno. LinkedList me random element paane ke liye traverse karna padta hai (O(n)). Plain array me front se nikalne pe baaki sab shift karne padte hain (O(n)). Solution: array ko "gol" (circular) treat karo.*

RING BUFFER KA IDEA

`head` (front) aur `tail` (end) indices modulo ke through ghoomti rehti hain — array ka end aane pe wapas 0 pe aa jaati hain.

- Enqueue → tail pe daalo, `tail = (tail+1) % cap`
- Dequeue → head se lo, `head = (head+1) % cap`
- Random → `(head + rand(0, size-1)) % cap` — seedha index access
- Full ho jaaye → capacity double karo (isliye **amortized** O(1))

STEP 2 — NORMAL (SINGLE-THREAD) SOLUTION

PEHLE BINA LOCK KE LIKHO

```
class RandomQueue<T> {
    private Object[] arr;
    // Object[] isliye, kyunki Java me `new T[cap]` allowed nahi hai
    // (generics runtime pe erase ho jaate hain - "type erasure").
    // Isliye Object[] rakhte hain aur nikalte waqt cast karte hain.
    private int head = 0, tail = 0, size = 0, capacity;
    private final Random rand = new Random();

    public RandomQueue(int cap) {
        this.capacity = cap;
        this.arr = new Object[cap];
    }

    public void enqueue(T val) {
        if (size == capacity) resize(capacity * 2);
        arr[tail] = val;
        tail = (tail + 1) % capacity;   // end pe pahuncha to wapas 0 pe
        size++;
    }

    @SuppressWarnings("unchecked")
    // Ye annotation compiler ka "unchecked cast" warning chhupata hai.
    // Hum jaante hain ki andar sirf T hi daala gaya hai, isliye cast safe
hai.
    public T dequeue() {
        if (size == 0) throw new NoSuchElementException();
        T val = (T) arr[head];
        arr[head] = null;               // reference hatao taaki GC memory
chhod sake
        head = (head + 1) % capacity;
        size--;
        return val;
    }

    @SuppressWarnings("unchecked")
    public T random() {
```

```
        if (size == 0) throw new NoSuchElementException();
        int idx = (head + rand.nextInt(size)) % capacity;
        // head se 0..size-1 ka random offset, phir modulo se wrap
        return (T) arr[idx];
    }

    private void resize(int newCap) {
        Object[] newArr = new Object[newCap];
        for (int i = 0; i < size; i++)
            newArr[i] = arr[(head + i) % capacity];  // logical order me
copy
        arr = newArr; head = 0; tail = size;         // ab seedha 0 se shuru
        capacity = newCap;
    }
}
```

DRY RUN (CAP 4)

Step	Action	head	tail	size	Array
1	enq(A)	0	1	1	[A,_,_,_]
2	enq(B)	0	2	2	[A,B,_,_]
3	deq()→A	1	2	1	[_,B,_,_]
4	enq(C)	1	3	2	[_,B,C,_]

STEP 3 — AB THREAD-SAFE BANAO

KYA CHEEZ SHARED AUR MUTABLE HAI?

`arr`, `head`, `tail`, `size` — chaaron threads ke beech share bhi hote hain aur badalte bhi hain. Yani **shared mutable state**. Sudoku wali problem se ulta case hai — yahan lock chahiye hi chahiye.

CONCEPT BREAK — REENTRANTLOCK VS SYNCHRONIZED

```
// Do options hain:
//
// (a) synchronized  -> simple, JVM khud lock release karta hai (exception
pe bhi)
// (b) ReentrantLock -> zyada control:
//        tryLock(timeout)     - lock na mile to wait mat karo (deadlock se
bachne ka tarika)
//        lockInterruptibly()  - wait ke beech thread interrupt ho sakta hai
//        fairness policy      - jo pehle aaya usko pehle lock
//
// "Reentrant" ka matlab: jis thread ne lock liya hai wo usi lock ko
```

```
// dobara le sakta hai bina khud ko block kiye (nested method calls ke liye
zaroori).
//
// Golden rule: unlock() HAMESHA finally me. Bhool gaye aur exception aayi
to
// lock kabhi release nahi hoga - poora system hang.
```

THREAD-SAFE VERSION

```
class ConcurrentRandomQueue<T> {
    private Object[] arr;
    private int head = 0, tail = 0, size = 0, capacity;
    private final ReentrantLock lock = new ReentrantLock();
    private final Random rand = new Random();

    public ConcurrentRandomQueue(int cap) {
        this.capacity = cap;
        this.arr = new Object[cap];
    }

    public void enqueue(T val) {
        lock.lock();
        try {
            if (size == capacity) resize(capacity * 2);
            arr[tail] = val;
            tail = (tail + 1) % capacity;
            size++;
        } finally {
            lock.unlock();   // finally me - hamesha
        }
    }

    @SuppressWarnings("unchecked")
    public T dequeue() {
        lock.lock();
        try {
            if (size == 0) throw new NoSuchElementException();
            T val = (T) arr[head];
            arr[head] = null;
            head = (head + 1) % capacity;
            size--;
            return val;
            // Dhyan do: return finally se PEHLE hota hai dikhne me,
            // lekin JVM finally block chalane ke baad hi return karta hai.
            // Isliye unlock() guaranteed chalega.
        } finally {
            lock.unlock();
        }
    }
```

```
    @SuppressWarnings("unchecked")
    public T random() {
        lock.lock();
        try {
            if (size == 0) throw new NoSuchElementException();
            int idx = (head + rand.nextInt(size)) % capacity;
            return (T) arr[idx];
            // random() sirf padh raha hai, phir bhi lock kyun?
            // Kyunki padhte waqt koi doosra thread resize() kar sakta hai
            // aur hum ek purane/aadhe-copy array se galat value uthayenge.
        } finally {
            lock.unlock();
        }
    }

    private void resize(int newCap) {
        // NOTE: ye private hai aur sirf enqueue() ke andar se call hota
hai,
        // jahan lock already hold kiya hua hai. Isliye yahan dobara lock
nahi liya.
        Object[] newArr = new Object[newCap];
        for (int i = 0; i < size; i++)
            newArr[i] = arr[(head + i) % capacity];
        arr = newArr; head = 0; tail = size;
        capacity = newCap;
    }
}
```

STEP 4 — TRAPS AUR POORE ANSWERS

INTERVIEW TRAP

`resize()` sabse risky operation hai — poora array copy hota hai. Ye bolna zaroor: "resize ke dauraan koi doosra thread access nahi kar sakta, isliye ye lock ke andar hi hona chahiye. Aur isi wajah se complexity **amortized** O(1) hai, strict O(1) nahi — kyunki kabhi-kabhi ek operation O(n) ka ho jaata hai."

Q1: random() O(1) kaise hai jabki head kahin bhi ho sakta hai?

A: Underlying structure ek array hai, isliye kisi bhi index tak seedha pahunch sakte hain. Bas `rand.nextInt(size)` se 0..size-1 ka offset lo, use head me add karo, aur modulo se wrap kar do. Koi traversal nahi — ek arithmetic aur ek array access, dono O(1).

Q2: Amortized O(1) ka matlab exactly kya hai?

A: Zyadatar enqueue O(1) hain, lekin jab array full hota hai to ek enqueue O(n) ka ho jaata hai (poora copy). Kyunki hum capacity **double** karte hain, ye mehnga operation

exponentially kam hota jaata hai — n operations ka total kaam O(n) rehta hai, isliye per-operation average O(1). Agar capacity +1 karke badhate, to har enqueue copy karta aur amortized O(n) ho jaata.

Q3: Ek hi lock ke bajaye enqueue/dequeue ke liye alag locks nahi le sakte?

A: Theoretically haan — head aur tail alag ends hain, isliye `LinkedBlockingQueue` jaisa "two-lock" design possible hai (putLock aur takeLock alag). Lekin yahan do dikkat hain: (1) `size` dono taraf badalta hai, isliye use atomic banana padega, (2) `resize()` poore array ko chhoota hai, isliye usme dono locks chahiye. Interview me bolne layak jawab: "Main single lock se shuru karunga; agar profiling me contention dikhe to two-lock ya lock-free (CAS-based) design pe jaunga."

Q4: ReentrantLock ki jagah synchronized use karte to?

A: Kaam kar jaata aur code chhota hota. ReentrantLock isliye chuna kyunki wo `tryLock(timeout)` aur `lockInterruptibly()` deta hai — production me agar koi thread lock ke liye anant tak wait na kare, ye kaam aata hai. Agar aisi zaroorat nahi hai to `synchronized` hi behtar hai, kyunki usme unlock bhoolne ka risk hi nahi hota.

CHAPTER 02 · DONO CANDIDATES KO AAYA THA

LRU Cache — Saari Variations

Naive → HashMap+DLL → TTL → Heaps only → Thread-safe. Har step pe kya badla aur kyun.

4 LRU Classic — Base Version ★

STEP 1 — PROBLEM SAMJHO

Cache me limited jagah hai (capacity N). Jab full ho jaaye aur naya item aaye, **sabse purana-use-kiya-gaya** *(Least Recently Used) item nikaal do.* `get` *aur* `put` *dono O(1) me hone chahiye.*

STEP 2 — NAIVE SOLUTION AUR WO KYUN FAIL HOTI HAI

ATTEMPT 1 — SIRF EK LIST

Items ko ArrayList me rakho, har access pe usko dhundh ke front me le aao. Problem: dhundhne ke liye poori list traverse karni padegi → `get()` O(n).

ATTEMPT 2 — SIRF EK HASHMAP

Lookup to O(1) ho gaya, lekin ab "kaun sabse purana hai" ka koi record hi nahi hai. Evict karte waqt poora map scan karna padega → O(n).

Seekh: humein **do** alag cheezein chahiye — fast lookup aur recency order. Isliye do data structures.

STEP 3 — OPTIMAL: HASHMAP + DOUBLY LINKED LIST

KAUN KYA KAREGA

- **HashMap** → key se node turant milna, O(1) lookup
- **Doubly Linked List** → recency order; head ke paas = recently used, tail ke paas = LRU (yahi evict hoga)

DLL kyun, singly nahi? Node ko beech se hatane ke liye uske **previous** node ka pointer chahiye (taaki `prev.next` ko aage jodo). Singly list me previous dhundhne ke liye shuru se traverse karna padega — O(n) ho jaayega.

POORA CODE

```
class LRUCache {
    class Node {
        int key, value;      // key BHI store karni hai - kyun? niche trap
dekho
        Node prev, next;
        Node(int k, int v) { key = k; value = v; }
    }

    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head, tail;   // dummy sentinels (asli data nahi)
    private final int capacity;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head.next = tail;    // shuru me list khaali hai: head <-> tail
        tail.prev = head;
    }

    public int get(int key) {
        Node node = map.get(key);
        if (node == null) return -1;
        moveToFront(node);   // ab ye "recently used" ho gaya
        return node.value;
    }

    public void put(int key, int value) {
        Node node = map.get(key);
        if (node != null) {          // already hai - sirf update karo
            node.value = value;
            moveToFront(node);
            return;
        }
        if (map.size() == capacity) {   // jagah nahi - LRU nikaalo
            Node lru = tail.prev;       // tail se just pehle wala = sabse
purana
            remove(lru);
            map.remove(lru.key);        // yahan node.key chahiye tha!
        }
        Node fresh = new Node(key, value);
        map.put(key, fresh);
        addFirst(fresh);
    }

    private void remove(Node n) {
        n.prev.next = n.next;    // pichhle wale ko aage wale se jodo
        n.next.prev = n.prev;
    }
```

```
    private void addFirst(Node n) {
        n.next = head.next;      // naya node head ke turant baad
        n.prev = head;
        head.next.prev = n;
        head.next = n;
    }

    private void moveToFront(Node n) {
        remove(n);
        addFirst(n);
    }
}
```

DUMMY HEAD/TAIL KA FAYDA

Inke bina har remove/add me "agar ye pehla node hai to...", "agar aakhri hai to..." wale null-checks likhne padte. Sentinels se har node ka `prev` aur `next` hamesha non-null rehta hai, code clean aur bug-free ho jaata hai. Ye baat interview me bolna — senior signal hai.

STEP 4 — TRAPS AUR POORE ANSWERS

SABSE COMMON BUG

Node ke andar **key bhi store karni padti hai**, sirf value nahi — warna evict karte waqt tail se node to mil jaayega, par map se kaunsi key hataani hai ye pata hi nahi chalega.

Complexity: get O(1), put O(1). Space O(capacity).

Q1: Node me key store karna kyun zaroori hai, value to hai hi?

A: Eviction ke waqt hum `tail.prev` se node uthate hain. Us node ko DLL se hata dena aasan hai, lekin `map` se bhi hataana hai — aur map key se index hota hai, node se nahi. Agar node ke andar key nahi hai, to map me us node ki key dhundhne ke liye poora map scan karna padega — O(n). Key store karne se `map.remove(node.key)` O(1) me ho jaata hai.

Q2: Java me `LinkedHashMap` to already ye karta hai, use kyun nahi karte?

A: Bilkul karta hai — `new LinkedHashMap<>(cap, 0.75f, true)` me teesra parameter `accessOrder=true` lagane se access order maintain hota hai, aur `removeEldestEntry()` override karke eviction mil jaata hai. Production me yahi use karna chahiye. Lekin interview me wo tumse **internals** dekhna chahte hain, isliye manual HashMap+DLL likhna hai. Ye jawab bolna acha lagta hai: "Production me main LinkedHashMap use karta, lekin yahan main internals dikhane ke liye khud implement kar raha hoon."

Q3: get() ke liye HashMap ka bucket collision worst case O(n) hai, to O(1) kaise bola?

A: HashMap ka O(1) **average case** hai. Worst case (sab keys ek hi bucket me) purane Java me O(n) tha; Java 8+ se ek bucket me 8 se zyada entries hone pe wo linked list se balanced tree me convert ho jaata hai, isliye worst case O(log n) hai. Practically achhi hash function ke saath O(1) hi maana jaata hai. Ye nuance bolna interview me alag dikhaata hai.

5 LRU Variation A — TTL ke saath

STEP 1 — KYA BADLA

Ab har entry ki ek **expiry time (TTL)** *bhi hai. Matlab entry do wajah se ja sakti hai — (1) capacity full hone pe LRU evict, ya (2) time khatam hone pe expire. Do alag "orders" ho gaye:* **recency order** *(DLL) aur* **expiry order** *(alag se track karna padega).*

STEP 2 — SIMPLE SOLUTION: LAZY EXPIRY

SABSE SIMPLE, O(1)

Node me `expiryTime` daal do. `get()` ke waqt check karo — expired ho to nikaal do aur -1 return karo.

```
class Node {
    int key, value;
    long expiryTime;    // naya field: epoch millis me kab expire hoga
    Node prev, next;
}

public int get(int key) {
    Node node = map.get(key);
    if (node == null) return -1;

    if (System.currentTimeMillis() > node.expiryTime) {
        remove(node);        // DLL se hatao
        map.remove(key);     // map se hatao
        return -1;           // expired = jaise mila hi nahi
    }
    moveToFront(node);
    return node.value;
}

public void put(int key, int value, long ttlMillis) {
    // ... classic put ka logic bilkul same ...
```

```
    fresh.expiryTime = System.currentTimeMillis() + ttlMillis;
}
```

STEP 3 — COMPLEXITY BADHAO: LAZY EXPIRY KA PROBLEM

INTERVIEWER YE ZAROOR POOCHHEGA

Agar koi expired key kabhi access hi na ho, to wo memory me padi rahegi aur capacity block karti rahegi — ek valid entry ki jagah kha jaayegi. Ye khud se bolo, phir solutions do.

TEEN BEHTAR SOLUTIONS

1. **Background cleanup thread** — ek `ScheduledExecutorService` har X second chal ke expired entries saaf kare. Simple, but poore map pe scan karna pad sakta hai (O(n) har sweep).
2. **Min-heap by expiryTime** — ek `PriorityQueue<Node>` jisme sabse jaldi expire hone wala top pe ho. Cleanup me sirf top dekho — top expired nahi to koi bhi expired nahi. Insert O(log n), cleanup bahut efficient.
3. **Eviction ke waqt bhi check** — jab capacity full ho, pehle tail se koi expired node dhundh lo aur usko nikaalo (valid LRU node bachane ke liye).

Interview framing: "Lazy expiry se shuru karunga kyunki simple aur O(1) hai, phir agar memory ek concern hai to min-heap ya background sweeper add karunga. Trade-off hai simplicity vs memory-efficiency."

STEP 4 — Q&A

Q1: Min-heap approach me cleanup fast kyun hai?

A: Heap ka invariant hai ki top hamesha sabse chhota element hota hai — yahan matlab sabse jaldi expire hone wala. To cleanup me sirf top dekho: agar top abhi expired nahi hua hai, to baaki koi bhi expired nahi ho sakta (kyunki sabki expiry usse baad ki hai) — turant ruk jao. Poora map scan karne ki zaroorat hi nahi. Sirf expired entries hi pop karni padti hain, har ek O(log n).

Q2: `System.currentTimeMillis()` use karna safe hai?

A: TTL ke liye theek hai, lekin ye **wall-clock** time hai — NTP sync ya manual change se peeche ja sakta hai, jisse entries galat time pe expire hongi. Duration measure karne ke liye `System.nanoTime()` behtar hai kyunki wo monotonic hai (kabhi peeche nahi jaata). Production cache me aksar ek monotonic clock abstraction rakhte hain, jo testing me bhi mock karna aasan bana deta hai.

Q3: Sliding TTL chahiye to (har access pe expiry reset)?

A: `get()` me `moveToFront()` ke saath `node.expiryTime = now + ttl` bhi set kar do. Iska matlab: jo entry actively use ho rahi hai wo kabhi expire nahi hogi — session management me yahi chahiye hota hai. Trade-off: ek popular entry hamesha zinda reh sakti hai, isliye kabhi-kabhi absolute max-lifetime bhi saath me rakhte hain.

6 LRU Variation B — Heaps + Maps (DLL allowed nahi)

STEP 1 — YE QUESTION KYUN POOCHHA JAATA HAI

Doosre candidate ko LRU **heaps + maps** *se implement karne ko bola gaya — DLL approach mana thi. Ye deliberately tricky hai: interviewer dekhna chahta hai ki tum trade-offs samajhte ho ya sirf ratta maara pattern likh rahe ho.*

STEP 2 — SOLUTION

IDEA

Har entry ke saath ek **monotonic counter** rakho (kab last access hua). Min-heap us counter pe ordered — top pe sabse purana. Evict karna ho to heap ka top nikaal do.

```
class Entry {
    int key, value;
    long accessCounter;   // timestamp ke bajaye counter kyun?
                          // Kyunki do access ek hi millisecond me ho sakte
hain
                          // aur tab order ambiguous ho jaata. Counter
hamesha unique.
}

class LRUCacheHeap {
    private final Map<Integer, Entry> map = new HashMap<>();
    private final PriorityQueue<Entry> heap =
        new PriorityQueue<>((a, b) ->
            Long.compare(a.accessCounter, b.accessCounter));
        // comparator: chhota counter = purana = top pe (min-heap)
    private long counter = 0;
    private final int capacity;

    public int get(int key) {
        Entry e = map.get(key);
        if (e == null) return -1;
        heap.remove(e);              // O(n) - YAHI bottleneck hai
        e.accessCounter = ++counter;
        heap.offer(e);               // O(log n)
        return e.value;
```

```
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            Entry e = map.get(key);
            e.value = value;
            heap.remove(e);
            e.accessCounter = ++counter;
            heap.offer(e);
            return;
        }
        if (map.size() == capacity) {
            Entry lru = heap.poll();   // O(log n) - sabse purana nikla
            map.remove(lru.key);
        }
        Entry fresh = new Entry(key, value, ++counter);
        map.put(key, fresh);
        heap.offer(fresh);
    }
}
```

STEP 3 — YE VERSION DLL SE WORSE HAI, AUR YAHI BOLNA HAI

TRADE-OFF KHUD BOLO

`heap.remove(element)` O(n) hai — heap sirf **top** pe efficient hai, beech me kisi arbitrary element ko dhundhne ke liye poora array scan karna padta hai. Isliye `get()` O(n) ho gaya, jabki DLL version me O(1) tha. Agar tum ye trade-off khud se bol do, tum jeet gaye.

OPTIMIZATION — LAZY DELETION

Heap se remove mat karo. Bas nayi entry naye counter ke saath daal do, aur purani ko "stale" maano. Poll karte waqt check karo: heap ka top jo counter bata raha hai, wo map me us key ki current entry ke counter se match karta hai? Agar nahi, wo stale hai — discard karke agla poll karo. Isse `get()` O(log n) ho jaata hai (O(n) ke bajaye), memory thodi zyada lagti hai.

Interview me bolna: "Heap-based LRU possible hai, lekin get() O(log n) ya O(n) ho jaata hai jabki DLL+HashMap se O(1) milta hai. Main DLL prefer karunga, unless requirement me kuch aisa ho jo heap zaroori banaye — jaise TTL-based ordering bhi chahiye."

STEP 4 — Q&A

Q1: Heap-based LRU me sabse bada bottleneck kya hai?

A: `heap.remove(element)`. PriorityQueue andar se ek array-based binary heap hai. Wo sirf ye guarantee deta hai ki parent apne children se chhota hai — koi index nahi rakhta ki kaun sa element kahan hai. Isliye kisi specific element ko hataane ke liye pehle use linear search se dhundhna padta hai (O(n)), phir heap ko fix karna padta hai (O(log n)). Net O(n).

Q2: Kya is O(n) ko theek kiya ja sakta hai bina lazy deletion ke?

A: Haan — "indexed heap" / "decrease-key supported heap" banake. Ek extra `Map<key, heapIndex>` maintain karo jo batata rahe ki har element heap array me kahan hai, aur har swap pe ye map update karo. Tab `remove` O(log n) ho jaata hai. Lekin ye khud likhna padta hai — Java ki built-in PriorityQueue ye nahi deti. Interview me isko mention karna kaafi hai, poora likhne ki zaroorat nahi.

Q3: Lazy deletion me heap kitna bada ho sakta hai?

A: Worst case har `get()` ek stale entry chhod jaata hai, to heap size operations ki sankhya ke barabar badh sakta hai. Isliye ek threshold rakhte hain — jab `heap.size() > 2 * capacity` ho jaaye, to poora heap rebuild kar do sirf valid entries se. Ye rebuild O(n) hai but kabhi-kabhi hi hota hai, isliye amortized cost kam rehti hai.

7 Thread-Safe LRU (bahut likely follow-up)

Cache hamesha multi-threaded environment me chalta hai — ye follow-up aana lagbhag tay hai: "Ab isko thread-safe banao."

STEP 1 — SABSE SIMPLE (AUR IMANDAAR) JAWAB

Saare public methods pe ek hi lock lagao.

```
private final ReentrantLock lock = new ReentrantLock();

public int get(int key) {
    lock.lock();
    try { /* wahi purana logic */ }
    finally { lock.unlock(); }
}
```

Ye correct hai, lekin saare reads/writes serialize ho jaate hain — high-throughput cache me bottleneck.

TRAP — "READ KE LIYE READWRITELOCK USE KAR LO NA?"

Yahan **nahi chalega**, aur ye samajhna hi asli test hai. `get()` dikhne me read hai, lekin wo andar se `moveToFront()` karta hai — matlab DLL ko **modify** karta hai. To LRU ka "read" bhi actually ek write hai. ReadWriteLock ka fayda tabhi milta hai jab reads sach me pure ho.

STEP 2 — PRODUCTION-GRADE APPROACH

- **Sharding / striping** — cache ko N chhote LRU shards me todo, key ka hash decide kare kaunsa shard. Har shard ka apna lock. Alag-alag keys ke operations parallel chalenge. (ConcurrentHashMap bhi yahi karta hai.) Trade-off: eviction ab per-shard hoti hai, global-exact LRU nahi rehti — practically ye acceptable hota hai.
- **Recency ko approximate karo** — har `get()` pe DLL modify mat karo. Access events ko ek buffer me daalo aur baad me batch me apply karo. Caffeine library yahi karti hai. Isse reads lock-free ke kareeb ho jaate hain.
- **Ready-made** — production me Caffeine ya Guava Cache use karo, jo ye sab optimizations aur TTL bhi built-in dete hain.

Q: Sirf ConcurrentHashMap use kar lete to LRU thread-safe ho jaata?

A: Nahi. ConcurrentHashMap sirf map wale hisse ko safe karta hai — DLL abhi bhi unprotected rahegi, aur do threads ek saath `moveToFront()` karke pointers corrupt kar denge. Aur ek aur baari problem: "capacity check karo, phir evict karo" ek check-then-act sequence hai, jo do alag structures pe faila hua hai — use ek hi lock ke andar hona hi padega. Isliye map thread-safe hone se poora cache thread-safe nahi ho jaata.

CHAPTER 03

DSA + LLD Round

Design problems, parsing, aur file I/O — file-system ke concepts wahin samjhaye gaye hain jahan aate hain.

8 Session Manager (HashMap + TTL)

STEP 1 — PROBLEM

Login karne pe server ek **session ID** *deta hai. Har request pe ye ID aati hai, aur server ko turant batana hai — "ye kaun hai, valid hai ya expire ho gaya." Ye lookup lakhon baar hoti hai, isliye O(1) chahiye.*

STEP 2 — SIMPLE SOLUTION

```
class Session {
    String userId;
    long createdAt, expiresAt;
}

class SessionManager {
    private final Map<String, Session> sessions = new HashMap<>();

    public String createSession(String userId, long ttlMillis) {
        String sessionId = UUID.randomUUID().toString();
        // UUID.randomUUID() cryptographically strong random deta hai (122
bits).
        // Sequential ID kabhi mat use karo - koi bhi guess karke
        // doosre ka session hijack kar lega.
        long now = System.currentTimeMillis();
        sessions.put(sessionId, new Session(userId, now, now + ttlMillis));
        return sessionId;
    }
}
```

STEP 3 — THREADING ADD KARO

KYUN ZAROORI HAI

Web server har request alag thread pe handle karta hai — `sessions` map ko sainkdon threads ek saath chhuenge. Plain HashMap yahan corrupt ho sakta hai (purane Java me infinite loop tak ho jaata tha resize ke dauraan).

```
private final Map<String, Session> sessions = new ConcurrentHashMap<>();
// ConcurrentHashMap bucket-level locking karta hai - poora map lock nahi
hota,
// isliye alag-alag keys ke operations sach me parallel chalte hain.
// Collections.synchronizedMap() bhi thread-safe hai, par wo poore map pe
// ek hi lock lagata hai - isliye slow.

public boolean isValid(String sessionId) {
    Session s = sessions.get(sessionId);
    if (s == null) return false;
    if (System.currentTimeMillis() > s.expiresAt) {
        sessions.remove(sessionId);   // lazy expiry
        return false;
    }
    return true;
}

public void invalidate(String sessionId) {
    sessions.remove(sessionId);   // logout
}
```

STEP 4 — Q&A

Q1: Multi-server setup me in-memory session map kyun fail karta hai?

A: Load balancer ek user ki requests kisi bhi server pe bhej sakta hai. Server A pe bana session server B ki memory me nahi hota, isliye user ko random logout dikhega. Teen solutions: (1) **Shared store** — Redis me sessions rakho, sab servers wahin se padhein (extra network hop, but simple aur revoke karna aasan). (2) **Stateless JWT** — token khud signed data carry karta hai, server ko kuch store hi nahi karna padta; problem ye hai ki token revoke karna mushkil hai (short expiry + refresh token se manage karte hain). (3) **Sticky sessions** — load balancer ek user ko hamesha usi server pe bheje; kaam chalta hai but wo server marne pe session bhi gaya, aur scaling ajeeb ho jaati hai.

Q2: Expiry strategy kaunsi choose karoge?

A: Bilkul wahi trade-off jo LRU+TTL me tha. **Lazy** — O(1), simple, but expired sessions memory me pade rehte hain. **Background sweeper** — `ScheduledExecutorService` periodically saaf kare, memory theek rehti hai but sweep O(n). **Min-heap by expiry** — sabse efficient cleanup. Redis use kar rahe ho to sabse acha jawab: Redis ko hi TTL do

(`SET key val EX seconds`) — wo khud expire kar dega, humein kuch likhna hi nahi padega.

Q3: Sliding expiry chahiye to?

A: Har successful `isValid()` call pe `expiresAt = now + ttl` set kar do — active user ka session expire nahi hoga. Do dhyan ki baatein: (1) har request pe write hoga, jo Redis pe extra load hai — isliye aksar tabhi refresh karte hain jab TTL ka aadha se zyada beet chuka ho, (2) saath me ek absolute max-lifetime bhi rakho, warna session anant tak zinda reh sakta hai.

9 Time Window Map / Rate Counter

STEP 1 — PROBLEM

Track karna hai — "pichhle 5 minute me kitne events aaye?" Purane events apne-aap window se bahar ho jaane chahiye. Ye **rate limiting** *aur monitoring dashboards ka core pattern hai.*

STEP 2 — SIMPLE SOLUTION

```
class TimeWindowCounter {
    private final Deque<Long> timestamps = new ArrayDeque<>();
    // Deque = double-ended queue: dono taraf se O(1) add/remove.
    // ArrayDeque LinkedList se tez hai (better cache locality, kam
objects).
    private final long windowMillis;

    public void recordEvent() {
        long now = System.currentTimeMillis();
        timestamps.addLast(now);   // naya event hamesha end me
        cleanup(now);
    }

    public int countInWindow() {
        cleanup(System.currentTimeMillis());
        return timestamps.size();
    }

    private void cleanup(long now) {
        // Front se hi hataate hain kyunki hum hamesha time-order me
        // insert karte hain - to front hamesha sabse purana hota hai.
        while (!timestamps.isEmpty()
               && now - timestamps.peekFirst() > windowMillis) {
            timestamps.pollFirst();
```

```
        }
    }
}
```

STEP 3 — THREADING + MEMORY

THREAD SAFETY

Dono public methods pe `synchronized` laga do — `recordEvent` aur `countInWindow` dono deque ko modify karte hain (`countInWindow` bhi cleanup karta hai, isliye wo bhi write hai). ArrayDeque bilkul thread-safe nahi hai.

MEMORY TRAP

Agar har event ka timestamp store karoge aur traffic bahut high ho (millions/sec), Deque phat jaayega. **Better production answer — bucketing:** window ko chhote buckets me todo (5 min = 300 buckets of 1 sec), har bucket me sirf ek **count** rakho. Memory O(number of buckets) ho jaayegi — events ki sankhya se bilkul independent.

```
// Bucketing ka core: ek circular array of counters
AtomicLongArray buckets = new AtomicLongArray(300);  // 300 seconds
// event aaya:
int idx = (int)((now / 1000) % 300);
buckets.incrementAndGet(idx);
// count nikalna: saare buckets ka sum (300 additions - constant kaam)
// Purane bucket ko reuse karne se pehle uska timestamp check karke reset
karna padta hai.
```

STEP 4 — Q&A

Q1: Bucketing approach memory kaise bachata hai?

A: Har individual event ka timestamp store karne ke bajaye, per-bucket sirf ek counter rakhte hain. 1 million events ek hi second me aayein to bhi wo ek bucket me sirf ek integer hi lenge (value 1000000), jabki Deque me 1 million Long objects banti. Memory O(bucket count) ho jaati hai, O(event count) nahi.

Q2: Bucketing me accuracy ka kya nuksaan hai?

A: Bucket boundary pe resolution kho jaati hai. 1-second buckets ke saath tum ye nahi bata sakte ki us second ke andar event kis exact millisecond pe aaya — sabse purana bucket poora ya to andar hoga ya bahar. Isliye count zyada se zyada ek bucket ke worth galat ho sakta hai. Fix: chhote buckets lo (accuracy vs memory ka trade-off), ya **sliding window counter** use karo jo sabse purane bucket ka weighted (proportional) hissa leta hai.

Q3: Distributed rate limiting kaise karoge?

A: In-memory counter har server ka apna hoga, to 5 servers pe limit 5 guna ho jaayegi. Solutions: (1) **Redis counter** — `INCR + EXPIRE`, ya ek Lua script se atomically dono; sab servers ek hi counter dekhte hain. (2) **Token bucket in Redis** — burst allow karta hai aur smooth rate deta hai. (3) **Local + periodic sync** — har server local count rakhe aur periodically share kare; fast hai but approximate. Interview me bolna: "Strict limit chahiye to centralized Redis, throughput priority hai to local approximate."

10 Number of Atoms (LeetCode 726, Hard)

STEP 1 — PROBLEM

Formula string di hai jaise `K4(ON(SO3)2)2`. Har element ka total count nikaalo, output alphabetically sorted. Nested brackets ke andar ka sab kuch bahar wale multiplier se multiply hota hai.

INTUITION

Ye nested folders jaisa hai. Har `(` pe naya "scope" (map) kholo. Har `)` pe scope band karo, uske saare counts ko multiplier se multiply karo, aur bahar wale scope me merge kar do.

STEP 2 — SOLUTION

```
public String countOfAtoms(String formula) {
    int n = formula.length(), i = 0;
    Deque<Map<String, Integer>> stack = new ArrayDeque<>();
    stack.push(new HashMap<>());   // outermost scope

    while (i < n) {
        char ch = formula.charAt(i);

        if (ch == '(') {
            stack.push(new HashMap<>());   // naya nested scope
            i++;

        } else if (ch == ')') {
            Map<String, Integer> closed = stack.pop();
            i++;
            int start = i;
            while (i < n && Character.isDigit(formula.charAt(i))) i++;
            int mult = (start == i) ? 1     // ")" ke baad koi digit nahi =
1
```

```
                : Integer.parseInt(formula.substring(start, i));

            Map<String, Integer> parent = stack.peek();
            for (Map.Entry<String, Integer> e : closed.entrySet())
                parent.merge(e.getKey(), e.getValue() * mult, Integer::sum);
                // merge(key, value, fn): key nahi hai to value daal do,
                // hai to purani aur nayi value pe fn lagao (yahan sum).
                // Ye "get karo, null check karo, put karo" ka short form
hai.

        } else {
            // Element ka naam: ek uppercase + zero ya zyada lowercase (He,
Mg)
            int start = i;
            i++;
            while (i < n && Character.isLowerCase(formula.charAt(i))) i++;
            String element = formula.substring(start, i);

            int numStart = i;
            while (i < n && Character.isDigit(formula.charAt(i))) i++;
            int count = (numStart == i) ? 1   // koi digit nahi = 1
                : Integer.parseInt(formula.substring(numStart, i));

            stack.peek().merge(element, count, Integer::sum);
        }
    }

    // TreeMap = keys automatically sorted. Output alphabetical chahiye tha.
    TreeMap<String, Integer> sorted = new TreeMap<>(stack.pop());
    StringBuilder sb = new StringBuilder();
    for (Map.Entry<String, Integer> e : sorted.entrySet()) {
        sb.append(e.getKey());
        if (e.getValue() > 1) sb.append(e.getValue());  // count 1 likhna
nahi hai
    }
    return sb.toString();
}
```

DRY RUN — K4(ON(SO3)2)2

Step	Action	Stack (top→bottom)
1	K4	[{K:4}]
2	(	[{}, {K:4}]
3	O, N	[{O:1,N:1}, {K:4}]
4	(	[{}, {O:1,N:1}, {K:4}]
5	S, O3	[{S:1,O:3}, {O:1,N:1}, {K:4}]

6	)2 → merge	[{O:7,N:1,S:2}, {K:4}]
7	)2 → merge	[{K:4,O:14,N:2,S:4}]

STEP 3 — TRAPS AUR Q&A

SABSE COMMON BUG

Multiplier **poore closed map** pe lagta hai — jisme nested se already-merged counts bhi shamil hain. Step 6 me O already 7 tha (1 direct + 6 nested se), aur poora 7 hi ×2 hua. Sirf direct elements pe multiply karna galat hai.

Complexity: Time O(n) single pass (worst case $O(n^2)$ agar deeply nested merges bahut ho), Space O(n).

Q1: Stack kyun, recursion se nahi kar sakte?

A: Kar sakte ho — recursion aur explicit stack same cheez hain, bas recursion me stack JVM sambhalta hai. Explicit stack better hai kyunki (a) bahut deep nesting pe StackOverflowError nahi aayega, (b) index `i` ko aage badhana explicit stack me clean rehta hai, recursion me use return karna ya field banana padta hai.

Q2: Edge cases kaunse test karoge?

A: (1) Single element bina count — `"H"` → `"H"`. (2) Do-akshar wala element — `"Mg"`, taaki lowercase handling test ho. (3) Bracket bina multiplier — `"(H2O)"` → mult=1. (4) Nested bracket — `"((H)2)3"` → H6. (5) Same element alag jagah — `"H2OH"` → H3O. (6) Bade counts jahan multiplication overflow kar sakti hai — long use karne ki baat bolo.

Q3: `merge()` ki jagah kya likhte agar wo na hoti?

A: `parent.put(key, parent.getOrDefault(key, 0) + value)`. `merge` zyada saaf hai aur ConcurrentHashMap me atomic bhi hota hai — yahan single-threaded hai isliye dono barabar hain, lekin habit achhi hai.

11 tail -f Implementation

STEP 1 — PROBLEM

Linux ka `tail -f logfile` file ke end pe "chipka" rehta hai — naye lines likhte hi turant dikhata hai, bina poori file dobara padhe.

CONCEPT BREAK — FILE PADHNE KE TAREEKE

```
// Java me file padhne ke do alag mizaaj hain:
//
// 1. STREAM (BufferedReader / FileInputStream)
//    - sirf aage badh sakte ho, peeche nahi
//    - internally chhota buffer rakhta hai, memory constant
//    - poori file line-by-line process karne ke liye best
//
// 2. RANDOM ACCESS (RandomAccessFile)
//    - file ke kisi bhi byte-position pe seedha jump kar sakte ho
//    - file.seek(pos)          -> pointer wahan le jao
//    - file.getFilePointer()   -> abhi pointer kahan hai
//    - file.length()           -> current file size (LIVE - har call pe
fresh)
//    - "r" mode = read-only, "rw" = read-write
//
// tail -f ke liye RandomAccessFile chahiye, kyunki humein
// "pichhli baar jahan chhoda tha" wahan se dobara shuru karna hai.
```

STEP 2 — NAIVE SOLUTION AUR USKI PROBLEM

NAIVE

Har 500ms pe poori file padho aur last N lines print karo. **Problem:** log file GBs ki hoti hai — har poll pe poori file padhna O(file size), CPU aur disk dono barbaad.

STEP 3 — OPTIMAL: OFFSET TRACK KARO

```
public void tail(String filePath) throws Exception {
    RandomAccessFile file = new RandomAccessFile(filePath, "r");
    long filePointer = file.length();
    // End se shuru karte hain, kyunki tail -f purani lines nahi dikhata -
    // sirf ab se aage jo likha jaayega wo dikhata hai.

    while (true) {
        long currentLength = file.length();

        if (currentLength < filePointer) {
            // File CHHOTI ho gayi = truncate ya rotate hui hai
            // (logrotate ne purani file ko rename karke nayi khaali bana
di).
            // Ab shuru se padhna hoga.
            filePointer = 0;
        }

        if (currentLength > filePointer) {
            file.seek(filePointer);           // wahin jao jahan chhoda tha
            String line;
```

```
            while ((line = file.readLine()) != null)
                System.out.println(line);      // sirf NAYA hissa
            filePointer = file.getFilePointer();  // naya nishaan lagao
        }

        Thread.sleep(500);   // polling interval
    }
}
```

STEP 4 — FOLLOW-UPS AUR POORE ANSWERS

Q1: Poori file baar-baar kyun nahi padhte?

A: File GBs ki ho sakti hai. Offset yaad rakh ke sirf naya delta padhna O(naya data) hai, O(poori file) nahi. Ek 10GB log file me agar 100 bytes ki nayi line aayi, to hum sirf 100 bytes padhte hain.

Q2: Polling ke bajaye kuch better hai?

A: Haan — OS-level file events. Linux me **inotify**, macOS me kqueue, Windows me ReadDirectoryChangesW. Java me ye sab `java.nio.file.WatchService` ke peeche abstract kiye gaye hain: directory register karo aur `ENTRY_MODIFY` events pe react karo. Fayda: CPU tabhi jagta hai jab sach me kuch badla, aur latency bhi kam (turant pata chalta hai, 500ms wait nahi). Nuksaan: macOS pe Java ka WatchService andar se polling hi karta hai (native nahi), aur network filesystems (NFS) pe events reliably nahi aate — wahan polling hi safe hai. Ye trade-off bolna acha lagta hai.

Q3: Log rotation kaise handle karoge?

A: Do case hain. (1) **Truncate** — file same rehti hai, size 0 ho jaata hai. Detect karna aasan: `currentLength < filePointer`, to pointer 0 pe reset kar do (upar code me handle hai). (2) **Rename + naya file** — logrotate `app.log` ko `app.log.1` bana ke naya `app.log` banata hai. Yahan humara open file handle abhi bhi **purani** file ko point kar raha hai (Linux me file handle inode se bandha hota hai, naam se nahi) — hum chup-chaap ek aisi file padhte rahenge jisme ab kuch likha hi nahi ja raha. Fix: periodically path ka **inode** check karo (`Files.readAttributes(path, BasicFileAttributes.class).fileKey()`) — agar wo badal gaya hai to purana handle band karo aur nayi file kholo.

Q4: Ek line aadhi likhi ho tab kya hoga?

A: Ho sakta hai hum us waqt padh lein jab writer ne line ka aadha hissa hi flush kiya ho — tab hume aadhi line milegi aur baaki agli poll me. Fix: `readLine()` ka result tabhi print karo jab wo newline pe khatam hua ho; aadha data ek buffer me rakho aur agli poll

me use jodo. Isiliye production tools (Filebeat, Fluentd) apne paas ek partial-line buffer rakhte hain.

12 Large File — bina pura load kiye

STEP 1 — PROBLEM

50 GB ki file, 8 GB RAM. `Files.readAllLines()` use kiya to **OutOfMemoryError** *— kyunki wo poori file ko ek List me utha leta hai. Solution:* **streaming** *— ek time pe thoda hissa.*

STEP 2 — CASE A: SEQUENTIAL PROCESSING

```
try (BufferedReader reader = new BufferedReader(new FileReader(path))) {
    // try-with-resources: is bracket ke andar jo resource banaya hai,
    // block khatam hote hi (exception aaye tab bhi) uska close() apne aap
    // call ho jaata hai. File handles limited hote hain - leak hui to
    // "Too many open files" error aata hai. Isliye ye syntax hamesha use
karo.
    //
    // BufferedReader kya karta hai: andar ~8KB ka buffer rakhta hai.
    // Disk se 8KB ek saath uthata hai, phir tumhe line-by-line deta hai.
    // Bina buffer ke har readLine() ek syscall karti - bahut slow.
    // Memory hamesha ~8KB, chahe file 50GB ki ho.

    String line;
    while ((line = reader.readLine()) != null) {
        process(line);   // ek line process, phir wo GC ke liye free
    }
}
```

Complexity: Time O(n), Space **O(1)** — file size se independent. Yehi bolna hai.

STEP 3 — CASE B: LAST N LINES (HARDER)

PROBLEM KYA HAI

Files forward-read ke liye bani hoti hain — koi index nahi hota ki "last 10 lines yahan se shuru hoti hain". Trick: `RandomAccessFile` se **end se peeche** ki taraf padho, newlines gino, jab tak N na mil jaayein.

```
public List<String> lastNLines(String path, int n) throws IOException {
    List<String> result = new ArrayList<>();

    try (RandomAccessFile file = new RandomAccessFile(path, "r")) {
```

```
        long pointer = file.length() - 1;   // aakhri byte se shuru
        StringBuilder sb = new StringBuilder();

        while (pointer >= 0 && result.size() < n) {
            file.seek(pointer);
            char c = (char) file.read();
            // file.read() ek byte padhta hai aur pointer khud aage badha
deta hai,
            // isliye har baar seek() karna padta hai.

            if (c == '\n') {
                if (sb.length() > 0) {
                    result.add(sb.reverse().toString());
                    // reverse() isliye: hum peeche se padh rahe hain,
                    // to characters ulte order me aa rahe hain.
                    sb.setLength(0);   // buffer clear (naya object nahi
banate)
                }
            } else {
                sb.append(c);
            }
            pointer--;
        }
        // Loop khatam hone pe aakhri line (jo file ke bilkul shuru me hai)
        // sb me pending reh sakti hai - use bhi add karo.
        if (sb.length() > 0 && result.size() < n)
            result.add(sb.reverse().toString());
    }

    Collections.reverse(result);   // hume peeche se mili thi, seedha karo
    return result;
}
```

YE ZAROOR BOLNA

"Space complexity O(1) hai — file size se **independent**." Sirf "time O(n)" bolna kaafi nahi; memory-independence hi wo cheez hai jo interviewer test kar raha hai.

OPTIMIZATION

Byte-by-byte padhna slow hai — har `read()` ek syscall hai. Production me 4KB/8KB ke **chunks** me padho (`file.read(byte[] buf)`) aur us buffer ke andar newlines count karo. 8KB chunk = 8000 syscalls ki jagah 1.

STEP 4 — Q&A

Q1: Last N lines ke liye start se kyun nahi padhte?

A: Start se padhein to poori file traverse karni padegi sirf ye jaanne ke liye ki last N kahan se shuru hui — O(file size). 50GB file pe ye minutes lega. End se peeche padhne me sirf utna kaam hota hai jitna N lines ke liye chahiye — typically kuch KB.

Q2: Multi-byte characters (UTF-8) ke saath byte-by-byte padhna kaam karega?

A: **Nahi**, aur ye ek acha nuance hai bolne ke liye. UTF-8 me ek character 1-4 bytes ka ho sakta hai. `(char) file.read()` ek byte ko char maan leta hai — Hindi, emoji, ya accented characters toot jaayenge. Newline (`\n`, byte 0x0A) safe hai kyunki UTF-8 me wo kisi multi-byte sequence ka hissa nahi ban sakta — isliye line **boundaries** sahi milengi, lekin line ka **content** galat decode hoga. Sahi tarika: bytes ko collect karo, order theek karo, phir `new String(bytes, StandardCharsets.UTF_8)` se decode karo.

Q3: File padhte waqt koi use likh raha ho to?

A: Linux me reads aur writes block nahi karte, isliye tumhe ek "torn" (aadha-likha) view mil sakta hai — jaise ek line jo abhi poori nahi hui. Isliye `file.length()` bhi har call pe badal sakti hai. Handle karne ke tareeke: ek snapshot length lo aur usi tak padho; ya partial line ko buffer me rakho jab tak newline na mile; ya file ko copy/snapshot karke padho agar consistency zaroori hai.

Q4: Aur bhi fast padhna ho to?

A: **Memory-mapped file** — `FileChannel.map()` se file ka hissa seedha virtual memory me map ho jaata hai, OS page-by-page load karta hai aur user-space copy bach jaati hai. Bade files pe random access ke liye bahut fast. Nuksaan: 32-bit address space ki limits, aur unmap ka control Java me acha nahi hai. Sequential processing ke liye BufferedReader hi kaafi hai — ye khud bolna ki "optimize tabhi karunga jab profiling bataye".

13 Map + Tree DSA — Pattern Guide

Ek candidate ko Round 2 me "Map + Tree based" question mila tha, jisme **complexity aur test cases pe explicit focus** *tha. Ye ek specific question nahi, ek* **pattern family** *hai.*

COMMON PATTERNS

- **Vertical order traversal** → `TreeMap<column, List<node>>`
- **Clone a tree/graph** → `Map<oldNode, newNode>` visited-tracking ke liye
- **Build tree from parent-child pairs** → `Map<id, Node>` se lookup
- **Path sum with prefix counts** → `Map<prefixSum, count>`

- **LCA with parent map** → `Map<child, parent>`
- **Subtree deduplication** → `Map<serializedSubtree, count>`

EXAMPLE — VERTICAL ORDER TRAVERSAL

```
public List<List<Integer>> verticalOrder(TreeNode root) {
    if (root == null) return new ArrayList<>();

    TreeMap<Integer, List<Integer>> map = new TreeMap<>();
    // TreeMap isliye: columns ko left-se-right SORTED order me chahiye.
    Queue<TreeNode> nodes = new LinkedList<>();
    Queue<Integer> cols  = new LinkedList<>();
    // Do parallel queues: node ke saath uska column number bhi chalta hai.
    // (Alternative: ek Pair class banao - dono theek hain.)

    nodes.offer(root); cols.offer(0);   // root column 0 pe

    while (!nodes.isEmpty()) {
        TreeNode node = nodes.poll();
        int col = cols.poll();

        map.computeIfAbsent(col, k -> new ArrayList<>()).add(node.val);
        // computeIfAbsent: key nahi hai to nayi list banao aur daalo,
        // hai to existing return karo. "get, null check, put" ka short
form.

        if (node.left  != null) { nodes.offer(node.left);  cols.offer(col -
1); }
        if (node.right != null) { nodes.offer(node.right); cols.offer(col +
1); }
        // BFS use kiya hai taaki ek column ke andar nodes
        // top-to-bottom order me aayein.
    }
    return new ArrayList<>(map.values());
}
```

Is round me complexity + test cases pe zor tha, isliye har solution ke baad ye 4 cheezein bolne ki aadat daalo: (1) Time complexity + kyun (2) Space complexity + kyun (3) Edge cases: null root, single node, skewed tree, duplicate values (4) Ek alternative approach aur uska trade-off.

Q1: Vertical order traversal me TreeMap kyun, HashMap kyun nahi?

A: Output me columns left-se-right sorted chahiye. TreeMap keys ko sorted rakhta hai (red-black tree), to `map.values()` seedha sahi order me aa jaata hai. HashMap se karte to end me keys nikaal ke sort karna padta — O(k log k) extra, aur ek extra step

jahan bug ho sakta hai. Trade-off: TreeMap ka har operation O(log k) hai vs HashMap O(1) — lekin k (columns ki sankhya) chhota hota hai, isliye ye kharcha na ke barabar hai.

Q2: BFS ki jagah DFS use karte to?

A: DFS bhi columns to sahi bhar dega, lekin ek column ke andar nodes ka order galat ho jaayega — DFS ek poori branch neeche tak jaata hai, isliye neeche wala node upar wale se pehle add ho sakta hai. Fix karne ke liye har node ke saath uski **row** bhi store karni padegi aur end me (row, insertion-order) pe sort karna padega. BFS level-by-line chalta hai isliye ye problem hoti hi nahi.

Q3: Edge cases kaunse test karoge?

A: (1) null root → khaali list. (2) Sirf ek node → ek hi column. (3) Left-skewed tree (sab left children) → har node apne alag column me, columns negative jaayenge. (4) Duplicate values → values same hone se kuch nahi bigadta kyunki hum position se index kar rahe hain, values se nahi. (5) Bahut deep tree → BFS me queue bada hoga (O(width)), DFS hota to stack overflow ka risk hota.

CHAPTER 04

HLD + Domain + Behavioral

Open-ended design, Kafka/KORA/Marketplace domain, aur STAR stories.

14 Requirement Gathering Framework

SABSE IMPORTANT INSIGHT

Dono candidates ne bataya HLD round **open-ended** tha. Interviewer ek line me problem dega ("design a key-value store") aur chup ho jaayega. Tera evaluation isi baat pe hoga ki **tu sahi sawaal poochta hai ya nahi** — turant solution likhna sabse badi galti hai.

PEHLE 5-7 MINUTE SIRF SAWAAL — 4 CATEGORIES

1. **Functional** — kya operations chahiye? (get/put/delete? range queries? transactions?)
2. **Scale** — kitne users? QPS? Data size? Read-heavy ya write-heavy?
3. **Non-functional** — latency target? availability (99.9% ya 99.99%)? consistency kitni strict?
4. **Constraints** — single datacenter ya multi-region? cost matter karta hai? existing tech stack?

Opening line: "Solution pe jaane se pehle main kuch requirements clarify karna chahunga, taaki main sahi trade-offs choose kar sakun. Pehla — is system ka primary use-case kya hai, aur expected scale kya hai?"

USKE BAAD KA FLOW

1. Requirements + scale estimate (numbers likho — QPS, storage) → 2. High-level architecture (boxes and arrows) → 3. Data model / API design → 4. Deep dive ek-do components me → 5. Bottlenecks + trade-offs → 6. Failure scenarios (ek node mar jaaye to?).

15 Design a Key-Value Store

Ye exact question ek candidate ko mila tha. Distributed KV store = DynamoDB/ Cassandra/Redis jaisa. Confluent ke liye extra relevant kyunki Kafka khud partitioning aur replication pe chalta hai — bilkul same concepts.

STEP 1 — REQUIREMENTS CLARIFY KARO

Operations (get/put/delete? range scan?), value size (KB ya MB?), scale (TB/PB? QPS?), consistency (strong ya eventual?), durability (data loss acceptable hai?).

STEP 2 — SINGLE NODE SE SHURU KARO

"Single machine pe ye ek HashMap + write-ahead log hai." Problem turant dikha do: (a) memory limit, (b) machine mari to sab gaya. Isliye distribute karna padega — incremental soch dikhao.

STEP 3 — PARTITIONING (SHARDING)

Naive: `hash(key) % N`. Problem: N badla (node add/remove) to **saari keys remap** ho jaati hain.

Solution — Consistent Hashing: nodes aur keys dono ko ek hash ring pe rakho; key us node pe jaati hai jo ring me clockwise sabse pehle mile. Node add/remove pe sirf padosi ka data move hota hai.

Virtual nodes: har physical node ko ring pe kai jagah rakho — load evenly distribute hota hai.

STEP 4 — REPLICATION + QUORUM

Har key N nodes pe (typically N=3), ring me agle N nodes pe. Quorum: W = kitne nodes pe likhna confirm ho, R = kitne se padhna. **W + R > N** ho to read me kam se kam ek latest copy guaranteed. N=3,W=2,R=2 balanced; W=1 fast writes weak consistency; W=3 slow but strong.

STEP 5 — CONSISTENCY (CAP)

Network partition hoga hi (P mandatory), to choose karo: **CP** (partition me kuch requests reject, data hamesha correct — banking) ya **AP** (hamesha respond, data thoda purana — social feed). Conflict resolution: **vector clocks** (kaunsi version kis se aayi track karo) ya **last-write-wins** (timestamp se, simple but data loss possible).

STEP 6 — STORAGE ENGINE

LSM Tree (Cassandra, RocksDB) — writes memory buffer me, phir sequentially disk pe flush; write-heavy ke liye fast. **B-Tree** — in-place updates; read-heavy ke liye better. Kafka bhi append-only log use karta hai — sequential writes hamesha random se tez.

STEP 7 — FAILURE HANDLING

Gossip protocol (nodes periodically ping karke health/membership share karte hain), **hinted handoff** (target down hai to koi aur node temporarily write rakhe, baad me

deliver kare), **Merkle trees** (replicas ka data compare karke sirf difference sync — anti-entropy repair).

Confluent-connection (bonus points): "Ye same concepts Kafka me bhi hain — partitions = sharding, replicas + ISR = replication, acks=all ≈ quorum write, aur log-structured append-only storage. KORA me ye multi-tenant scale pe extend hota hai."

Q1: Consistent hashing simple modulo se better kyun hai?

A: `hash(key) % N` me N (node count) badalte hi lagbhag **saari** keys ka target node badal jaata hai — 4 se 5 nodes karne pe ~80% data network pe move hoga, jo production me ghante lagata hai aur cluster ko down kar deta hai. Consistent hashing me nodes aur keys dono ek ring pe hote hain; naya node aane pe sirf uske aur uske padosi ke beech wali keys move hoti hain — roughly 1/N data. Virtual nodes isliye chahiye kyunki bina unke ring pe nodes unevenly baith jaate hain aur kuch nodes ko doosron se kai guna zyada load milta hai.

Q2: W + R > N kya guarantee deta hai?

A: Pigeonhole principle. Agar write W nodes pe successful hui aur read R nodes se hui, aur W+R > N, to in do sets me kam se kam **ek node common** hoga hi. Wo common node latest write dekh chuka hai, isliye read me wo latest version zaroor dikhega. Client versions compare karke naya wala chun leta hai. Dhyan do: ye "latest write dikh jaayegi" guarantee deta hai, lekin agar clients ke beech clock skew ho ya concurrent writes hon to conflict resolution (vector clocks) ki zaroorat phir bhi padti hai.

Q3: LSM tree write-heavy ke liye fast kyun hai?

A: LSM writes ko pehle memory (memtable) me leta hai aur disk pe sirf **sequential** flush karta hai. Disk — HDD ho ya SSD — sequential writes pe random writes se kai guna tez hai. B-tree me har update disk pe apni exact jagah dhundh ke likhna hota hai (random write) aur page splits bhi hote hain. Trade-off: LSM me reads dheere ho sakte hain kyunki ek key kai SSTables me ho sakti hai — isliye bloom filters aur compaction chahiye hote hain.

16 Kafka Core Internals — Quick Revision

Concept	Ek line me
Topic / Partition	Topic logical naam; partition actual ordered log. Order sirf partition ke andar guaranteed.
Replication / ISR	Har partition ki N copies. ISR = In-Sync Replicas — jo leader ke saath up-to-date hain.

acks	0 (fire-forget), 1 (leader ne likha), all (saare ISR ne likha — safest)
Consumer Group	Ek group me har partition sirf ek consumer ko. Parallelism = partition count.
Rebalancing	Consumer add/remove hone pe partitions dobara baante jaate hain. Stop-the-world hota hai.
Offset	Consumer kahan tak padh chuka. Commit karne pe hi progress save hoti hai.
Log Compaction	Har key ki sirf latest value rakho — changelog/state store ke liye.
Exactly-once	Idempotent producer + transactions se duplicate na ho.

Q1: Kafka ordering kaise guarantee karta hai?

A: **Sirf partition ke andar.** Ek partition ek append-only log hai, isliye usme messages ka order fix hai. Poore topic me ordering chahiye to do options: (a) single partition rakho — order milega but throughput ek broker tak seemit ho jaayega, (b) **key-based partitioning** — same key ke messages same partition me jaate hain, to per-key ordering mil jaati hai aur parallelism bhi bachi rehti hai. Zyadatar real systems ko sirf per-entity ordering chahiye hoti hai (jaise ek user ke events), isliye (b) hi sahi jawab hai.

Q2: Consumer group me parallelism ki limit kya hai?

A: Partition count. Agar topic me 6 partitions hain aur group me 10 consumers hain, to 6 ko ek-ek partition milegi aur 4 **bekaar baithe rahenge**. Isliye scaling ke liye pehle partitions badhane padte hain. Ulta bhi hota hai: bahut zyada partitions se rebalance slow ho jaata hai aur broker pe metadata overhead badhta hai.

Q3: acks=all lagane se data loss zero ho jaata hai?

A: Lagbhag, lekin akela kaafi nahi. `acks=all` ka matlab hai "saare **in-sync** replicas ne likha" — agar ISR sikud ke sirf leader tak reh gayi (baaki replicas peeche reh gaye), to acks=all bhi ek hi copy pe likhna ban jaata hai. Isliye saath me `min.insync.replicas=2` set karte hain, taaki itne replicas na hone pe write hi fail ho jaaye. Aur producer side pe retries + idempotence enable karna padta hai warna retry se duplicates aa jaate hain.

17 KORA — Confluent ka Cloud-Native Engine

Self-managed Kafka chalana ek full-time job hai — brokers, disk, scaling, recovery sab khud dekho. Confluent ne socha "customer ko Kafka **use** *karna chahiye,* **chalana** *nahi" — isi se bana* **KORA***, jo Apache Kafka ka lagbhag poora rewrite hai (~20% code hi original se match karta hai).*

CORE CONCEPTS

- **Logical Kafka Cluster** — customer jo dekhta/manage karta hai; peeche shared physical infra
- **CKU (Confluent Kafka Unit)** — capacity + billing ka unit
- **Control Plane vs Data Plane** — config/decisions alag, actual data movement alag
- **Multi-tenancy** — hazaaron customers ek shared cluster pe, safely isolated
- **Tiered Storage** — purana data cheap object storage pe, "infinite retention" bina cost explode kiye
- **Self-Balancing Clusters** — load khud redistribute hota hai, manual intervention nahi

18 Marketplace & Hub — Tera Actual Domain

Kafka ko Salesforce/MongoDB/Snowflake se connect karna ho to har baar khud connector likhna painful hoga. Isliye Confluent ek ***"App Store"*** *jaisa system chalata hai — pre-built connectors aur Flink artifacts browse karo, install karo, use karo.*

JD KE EXACT 4 RESPONSIBILITIES

1. **Catalog** — kaunse connectors/artifacts available hain, metadata, versions
2. **Publishing** — naya connector system me register karna
3. **Distribution** — customer ke Kafka/Flink environment tak deliver karna
4. **Lifecycle Management** — versions, deprecation, backward-compatibility

SCOPE CLEAR RAKHNA

Tu poore KORA broker-internals (replication protocol wagera) pe kaam nahi karega — tera scope **Marketplace & Hub ka ek well-defined portion** hai, jo KORA ecosystem ke andar ek component ki tarah deploy hota hai.

YE DOMAIN TERE CODING PROBLEMS SE KAISE JUDTA HAI (YE CONNECTION BOLNA)

- **Catalog service** = ek read-heavy lookup system → **LRU cache + TTL** bilkul yahi hai (connector metadata cache karo, version update pe invalidate karo)
- **Publishing pipeline** = versioned artifacts ka async processing → **Job Scheduler** pattern (validate, scan, package, publish — har step ek job)
- **Distribution** = artifacts ko hazaaron environments tak pahunchana → rate limiting aur retry → **Time Window Counter**
- **Lifecycle** = deprecation aur backward-compatibility → version comparison, schema evolution

Agar poochhein "is role me kya interest hai": "Mera background distributed systems aur Kafka pe hai — aur ye role usko ek platform/product perspective deta hai, jahan hazaaron customers ke liye artifacts ka lifecycle manage karna hota hai. Ye scale ka problem mujhe interesting lagta hai."

Q: "Connector catalog service design karo" aa jaaye to?

A: Same framework: requirements pehle. Functional — search/browse connectors, version list, metadata fetch, publish naya version. Scale — read-heavy hai (browse bahut, publish kam), isliye caching aggressive rakho. Data model — connector ID + version (immutable), metadata blob, artifact binary object storage me (S3), metadata DB me. Publishing — async pipeline (validate → security scan → package → index), status track karne ke liye job scheduler. Distribution — CDN ya regional object-storage replicas. Lifecycle — semantic versioning, deprecation flags, compatibility matrix. Key insight bolna: **artifacts immutable hone chahiye** — ek publish ki hui version kabhi badalti nahi, sirf nayi version aati hai. Isse caching trivial ho jaati hai aur reproducibility milti hai.

19 Behavioral Round + STAR

STAR

Situation — context (15 sec) → **T**ask — teri responsibility (15 sec) → **A**ction — tune kya kiya, specifically (60-90 sec, sabse bada hissa) → **R**esult — outcome, numbers ke saath (20 sec).

SABSE COMMON GALTI

"Hum ne ye kiya" bolna. Interviewer **tera** contribution sunna chahta hai — "**Maine** ye design kiya, **maine** ye decision liya." Team ka credit dena achha hai, but apna role clear rakhna.

5 STORIES TAYAAR RAKH (JD VALUES SE MATCH)

1. **Root cause tak gaya, symptom fix nahi kiya** — JD me literally likha hai. Koi production bug jise tune deep debug karke asli wajah dhundhi.
2. **Complex project quality ke saath deliver kiya** — EVA-AI ya koi bada feature, tests/logging/error-handling ke saath.
3. **Customer-focused decision** — customer impact ke liye extra mile gaya ya scope adjust kiya.
4. **Team ke liye decision, apne liye nahi** — apna preferred approach chhoda kyunki team/company ke liye behtar tha.

5. **Disagreement handle kiya** — technical disagreement jahan data se argue kiya, ego se nahi.

RECORDED VIDEO ASSESSMENT TIPS

Ek hi take milta hai — pehle likh ke practice kar, phir bolke. ~30 sec sochne ka time, phir ~3 min record. Camera me dekh, screen me nahi. Quiet + well-lit jagah, Chrome browser. Phone se khud ko record karke playback dekh — filler words pakad me aayenge.

Q: "AI tools ka use kaise karte ho, aur output verify kaise karte ho?"

A: Ye naya-zamane ka question hai aur tere paas EVA-AI ka production AI-integration experience hai — perfect fit. Structure: pehle batao kahan use karte ho (boilerplate, test cases, unfamiliar API explore karna, first draft). Phir **verify** ka hissa — yahi asli jawab hai: generated code ko line-by-line padhta hoon, tests likhta hoon jo main khud samajhta hoon, edge cases khud sochta hoon, aur jo cheez production me ja rahi hai uska logic khud validate karta hoon. Aakhir me EVA-AI ka concrete example do jahan tune AI output ko verify/guardrail kiya. Kabhi ye mat bolna ki tum AI output blindly trust karte ho.

FINAL

Cheat Sheet — Interview se Pehle

Sirf ye padh lo, sab wapas yaad aa jaayega.

Concurrency Problems

- Pehla step hamesha: shared mutable state identify karo
- Sudoku MT = 27 independent read-only tasks, no lock; AtomicBoolean for early exit; 9x9 pe sequential hi behtar — ye khud bolna
- Job Scheduler = PriorityBlockingQueue + dispatcher thread + worker pool; DelayQueue > polling; har job try-catch me
- Ring buffer = head/tail + modulo; resize lock ke andar; amortized O(1)
- unlock() hamesha finally me; volatile = visibility only; Atomic = visibility + atomicity (CAS)
- Deadlock fix: lock ordering, tryLock+timeout, chhota lock scope

LRU

- Classic = HashMap + DLL, dummy sentinels, Node me key BHI rakho, get/put O(1)
- DLL kyun: node hataane ke liye prev pointer chahiye
- +TTL = lazy expiry se shuru, phir min-heap/background sweeper ka trade-off bolo
- Heap version = worse hai (heap.remove O(n)) — ye khud bolna; lazy deletion se O(log n)
- Thread-safe = ek lock se shuru; ReadWriteLock kaam nahi karega kyunki get() bhi DLL modify karta hai; production me sharding ya Caffeine

DSA + LLD

- Session Manager = ConcurrentHashMap + TTL; multi-server pe Redis ya JWT
- Rate Counter = Deque of timestamps; high traffic pe bucketing (memory O(buckets))
- Number of Atoms = stack of maps; multiplier POORE closed map pe lagta hai
- tail -f = RandomAccessFile + offset; rotation pe inode check; WatchService > polling
- Large file = BufferedReader streaming (space O(1)); last-N = backward RandomAccessFile; chunks me padho
- Map+Tree = TreeMap jab sorted output chahiye; BFS jab level order matter kare

HLD + Domain + Behavioral

- Turant solution mat likho — 5-7 min requirements poochho

- Single node → phir scale; consistent hashing + virtual nodes; W+R>N; CAP me CP ya AP choose karo
- LSM (write-heavy) vs B-tree (read-heavy)
- Kafka: ordering sirf partition me; parallelism = partition count; acks=all + min.insync.replicas
- Marketplace = catalog (cache), publishing (async jobs), distribution (rate limit), lifecycle (versioning); artifacts immutable
- STAR: "Maine" bolo; root-cause story ready; AI-verification story ready

Interview Din Ka Checklist

- Har solution ke baad: time + space complexity khud se bolo, poochne ka wait mat karo
- Edge cases khud batao: null, empty, single-element, duplicates, overflow
- Thread-safety follow-up har coding question me aa sakta hai — ready raho
- Code likhne se pehle approach 30 sec me bolo, phir likho
- Atak jao to chup mat raho — loudly soch, interviewer hint dega